

Introduction to Unit Testing in Python

SE 413/513 — Software Testing

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,  
author = {Hasan, Kamrul},  
year = {2026},  
title = {Software Testing},  
institution = {Grand Valley State University},  
url = {https://github.com/mdkamrulhasan/software-testing}  
}
```

Contents

1	Learning Objectives	3
2	Introduction to Unit Testing	3
3	Why pytest?	4
4	Test Discovery and Running Tests	5
5	Assertions in pytest	5
5.1	Assertion Rewriting	5
5.2	Comparing Floating-Point Numbers	6
5.3	Testing for Exceptions	6
6	Fixtures: The Basics	7
6.1	Fixtures Can Depend on Other Fixtures	7
7	Setup and Teardown: Yield Fixtures	8
8	Fixture Scope	8
9	Useful Built-in Fixtures	9
9.1	tmp_path: A Temporary Directory	9
9.2	capsys: Capturing Printed Output	9
9.3	monkeypatch: Safely Patching Things	9
10	Parametrized Tests	10

11 Sharing Fixtures Across Files: <code>conftest.py</code>	11
12 Organizing a Growing Test Suite	12
12.1 Markers and Selective Test Runs	12
13 Common Pitfalls	13
14 A Practical Mental Model	13
15 Summary	14
16 Practice Exercises	15
17 Discussion Questions	15
18 Key Takeaways	16
19 References	16

1 Learning Objectives

After completing this reading, you should be able to:

- Define what a unit test is and explain the properties of a good unit test (FIRST).
- Explain why `pytest` is the de facto standard testing framework in the Python ecosystem.
- Write and run basic `pytest` tests using plain `assert` statements.
- Use `pytest.raises()` to test that code raises the correct exceptions.
- Structure a test using the Arrange-Act-Assert pattern.
- Create and use `pytest` fixtures, including `yield`-based setup/teardown.
- Choose an appropriate fixture scope (`function`, `class`, `module`, `session`).
- Use key built-in fixtures: `tmp_path`, `capsys`, and `monkeypatch`.
- Use `@pytest.mark.parametrize` to run one test against many inputs.
- Share fixtures across files with `conftest.py`.
- Organize a growing test suite using naming conventions, directory structure, and markers.

2 Introduction to Unit Testing

A **unit test** is a small, automated test that exercises one isolated piece of behavior in a program — typically a single function or method — and checks that it produces the correct result for a given input, without touching a real database, network, file system, or other parts of the application.^[2, 3] Unit testing is the lowest, most fine-grained level in the hierarchy of test levels (unit, integration, system, and acceptance testing) introduced in the *Testing Foundations* reading.

Keeping tests this small and isolated pays off in four concrete ways:

- **Fast feedback.** A test that exercises one function, with no real database or network call, runs in milliseconds. A suite of thousands of such tests can still finish in seconds, so a developer can re-run it after every small change instead of waiting minutes (or longer) for a full system test.
- **Precise fault localization.** When a unit test fails, the defect is narrowed down to the one function or method under test, rather than somewhere in an entire subsystem — exactly the fine-grained feedback a broader integration or system test cannot give you.
- **Regression safety.** Once written, a unit test keeps checking the same behavior on every future run, catching cases where a change made elsewhere in the code accidentally breaks something that used to work.
- **Executable documentation.** A well-named unit test records, in code, exactly what a function is supposed to do — documentation that cannot silently drift out of date the way a comment can, because it fails loudly the moment it stops being true.

Not every automated test that is small is automatically a *good* unit test. Five properties, often remembered by the acronym **FIRST**, distinguish a well-designed unit test from a poorly-designed one:^[4]

Property	Meaning
Fast	Runs in milliseconds, so it can be run constantly during development.
Isolated	Tests exactly one thing, and does not depend on other tests, or on the order tests happen to run in.
Repeatable	Produces the same result every time, in any environment — no dependence on network access, the system clock, or random data.
Self-checking	The test itself reports pass or fail; no human has to read output and judge whether it looks right.
Timely	Written close to the same time as the code it tests, not added as an afterthought weeks later.

Even with no testing framework at all, a unit test is nothing more than a small piece of code that calls a function and checks the result:

```
def add(a, b):
    return a + b

def test_add_two_positive_numbers():
    result = add(2, 3)
    assert result == 5    # fails loudly if add() is ever broken
```

This tiny example already has every property in the table above: it is fast, isolated, repeatable, self-checking (the `assert` decides pass or fail), and can be written at the same time as `add()` itself. What a framework adds is everything *around* this basic idea — automatically finding tests like this one across a whole project, reporting failures in a readable way, managing setup and teardown, and running thousands of them in an organized suite. That framework, for Python, is `pytest` — the subject of the rest of this reading.

Key Idea

A unit test is not defined by the tool that runs it, but by what it tests and how: one small piece of behavior, isolated from the rest of the system, checked automatically. `pytest` does not change that definition — it simply makes writing, running, and organizing unit tests in Python easier.

3 Why pytest?

Python ships with a built-in testing framework, `unittest`, which you have already seen in earlier readings. `unittest` works, but it borrows heavily from Java's JUnit: tests are methods on a class that inherits from `unittest.TestCase`, and comparisons go through a large family of specialized assertion methods (`assertEqual`, `assertTrue`, `assertIn`, `assertRaises`, and dozens more).

`pytest` is a third-party framework that has become the de facto standard for testing Python code, in industry and in open-source projects alike.^[5] It is not part of the standard library, so it must be installed:

```
pip install pytest
```

Two design decisions explain most of `pytest`'s popularity, and both are central to this reading:

1. **Plain `assert` statements.** You do not need to memorize a family of `assertSomething()` methods. You write ordinary Python `assert` expressions, and `pytest` produces detailed, readable failure output automatically.

2. **Fixtures.** Instead of rigid `setUp()/tearDown()` methods tied to a class hierarchy, **pytest** uses *fixtures*: small, composable, reusable functions that supply data, resources, or state to your tests through simple function parameters.

4 Test Discovery and Running Tests

pytest finds tests automatically, without any configuration, by following naming conventions:[5]

- Test files are named `test_*.py` or `*_test.py`.
- Test functions inside those files are named `test_*`.
- Test classes are named `Test*`, must not define an `__init__` method, and their test methods are also named `test_*`.

Given a file `test_pricing.py`:

```
def test_addition():
    assert 2 + 2 == 4
```

you can run it in several ways from the command line:

```
pytest                    # discover and run every test in the project
pytest test_pricing.py    # run everything in one file
pytest test_pricing.py::test_addition  # run one specific test
pytest -v                 # verbose: list each test by name
pytest -k "addition"      # run only tests whose name contains "addition"
```

Tip

Run **pytest** from your project's root directory as part of your normal workflow — ideally every time you save a file, and always before committing. A test suite you do not run regularly provides none of the early-defect-detection benefit described in the *Testing Foundations* reading's discussion of shift-left testing.

5 Assertions in pytest

A **pytest** test is, at its core, just a Python function that raises no exception if everything is fine, and raises an `AssertionError` (or some other exception) if something is wrong. The simplest possible test body is a single `assert` statement.

```
def calculate_discounted_price(price, discount_percent):
    return price * (1 - discount_percent / 100)

def test_ten_percent_discount():
    assert calculate_discounted_price(100, 10) == 90
```

5.1 Assertion Rewriting

What makes this more than a syntactic convenience is that **pytest** rewrites the bytecode of your test files at collection time so that a failing `assert` produces a detailed report of exactly what was compared — without you writing any special assertion method.[6] Suppose the function above had a fault:

```
def calculate_discounted_price(price, discount_percent):
    return price * (1 - discount_percent) # BUG: forgot to divide by 100
```

Running `pytest` against the test above produces output similar to:

```
def test_ten_percent_discount():
>     assert calculate_discounted_price(100, 10) == 90
E     assert -900.0 == 90
E     + where -900.0 = calculate_discounted_price(100, 10)
```

Notice that `pytest` tells you the actual computed value (`-900.0`), the expected value (`90`), and even which call produced the wrong value — all from one plain `assert`. This is the single biggest ergonomic difference from `unittest`, where the equivalent test requires remembering to call `self.assertEqual(...)` specifically (using `assertTrue` instead would report only “False is not true,” with no values shown).

5.2 Comparing Floating-Point Numbers

Because of how floating-point arithmetic works, direct equality comparisons between floats are risky:

```
>>> 0.1 + 0.2 == 0.3
False
```

`pytest` provides `pytest.approx()` for exactly this situation:[6]

```
import pytest

def test_float_addition():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

5.3 Testing for Exceptions

Some behavior is only correct if it raises the right exception. `pytest.raises()` is a context manager that asserts a specific exception type is raised inside its block:[6]

```
import pytest

def calculate_discounted_price(price, discount_percent):
    if not 0 <= discount_percent <= 100:
        raise ValueError("discount_percent must be between 0 and 100")
    return price * (1 - discount_percent / 100)

def test_invalid_discount_raises_value_error():
    with pytest.raises(ValueError):
        calculate_discounted_price(100, 150)
```

If the code inside the `with` block does *not* raise a `ValueError` (either because it raises nothing, or raises a different exception), the test fails — exactly the guarantee you want when testing error-handling paths.

Key Idea

Arrange–Act–Assert (AAA). Well-organized tests, in `pytest` or any framework, tend to follow the same three-part shape: **Arrange** the inputs and any needed state, **Act** by calling the code under test, and **Assert** that the outcome is correct. Skimming a test and being able to identify these three parts at a glance is a good sign the test is doing one clear thing.

6 Fixtures: The Basics

Real tests rarely operate on bare literals. They need data, configured objects, temporary files, database connections, or other setup — and that setup is often shared across many tests. `pytest` handles this with **fixtures**: functions decorated with `@pytest.fixture` that a test requests simply by naming it as a parameter.^[7]

```
import pytest

@pytest.fixture
def sample_cart():
    """Provide a shopping cart pre-loaded with two items."""
    return {
        "items": [
            {"name": "widget", "price": 9.99, "quantity": 3},
            {"name": "gadget", "price": 24.99, "quantity": 1},
        ]
    }

def cart_total(cart):
    return sum(item["price"] * item["quantity"] for item in cart["items"])

def test_cart_total(sample_cart):
    assert cart_total(sample_cart) == pytest.approx(54.96)
```

When `pytest` runs `test_cart_total`, it sees the parameter `sample_cart`, recognizes it as the name of a fixture, calls that fixture function, and passes its return value into the test. This is a form of **dependency injection**: the test declares what it needs, and `pytest` supplies it.

Tip

Because a fixture is just a function, multiple tests can request the same fixture, and each test gets it prepared fresh (by default). This avoids the classic problem of one test accidentally leaving behind state that corrupts the next test.

6.1 Fixtures Can Depend on Other Fixtures

Fixtures can themselves request other fixtures as parameters, letting you build up complex setups from small, composable pieces:

```
@pytest.fixture
def empty_cart():
    return {"items": []}

@pytest.fixture
def cart_with_one_item(empty_cart):
    empty_cart["items"].append({"name": "widget", "price": 9.99, "quantity": 1})
    return empty_cart

def test_single_item_total(cart_with_one_item):
    assert cart_total(cart_with_one_item) == pytest.approx(9.99)
```

7 Setup and Teardown: Yield Fixtures

Some resources need cleanup after a test finishes — closing a file, disconnecting from a database, deleting a temporary directory. A fixture that uses `yield` instead of `return` splits naturally into a setup half (before the `yield`) and a teardown half (after it):

```
import sqlite3
import pytest

@pytest.fixture
def db_connection():
    connection = sqlite3.connect(":memory:")          # setup
    connection.execute(
        "CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT)"
    )
    yield connection                                  # hand off to the test
    connection.close()                               # teardown

def test_insert_and_query_user(db_connection):
    db_connection.execute("INSERT INTO users (name) VALUES ('Alice')")
    db_connection.commit()
    row = db_connection.execute("SELECT name FROM users").fetchone()
    assert row[0] == "Alice"
```

The code after `yield` always runs after the test completes — **even if the test fails or raises an exception** — which makes yield fixtures the right tool for anything that must be cleaned up reliably.

Common Pitfall

A fixture with `return` instead of `yield` has no teardown phase at all. If you find yourself wanting to write cleanup code “after the test,” that is a signal you need a yield fixture, not a plain one.

8 Fixture Scope

By default, a fixture runs once *per test function* that requests it. For expensive setup — spinning up a real database, loading a large file, starting a server — re-running that setup for every single test can make a suite unbearably slow. The `scope` parameter controls how often a fixture is recreated.

Scope	Fixture is created...
function (default)	Once per test function. Safest: no state leaks between tests.
class	Once per test class.
module	Once per test file (module).
session	Once for the entire test run, shared across every file.

```
import pytest

@pytest.fixture(scope="module")
def db_connection():
    connection = sqlite3.connect(":memory:")
    connection.execute(
        "CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT)"
    )
    yield connection
```

```
connection.close()
```

With `scope="module"`, every test in this file shares the same database connection, created once, instead of paying setup/teardown cost for each test individually.

Common Pitfall

Wider scope is a performance optimization, not a free upgrade. A `session`-scoped fixture that holds *mutable* state (like the database above, where tests insert rows) can let one test's leftover data silently affect another test's result. This is a common source of **flaky tests** — tests that pass or fail unpredictably depending on execution order. If your fixture holds mutable state, either keep it `function`-scoped, or explicitly reset that state between tests (for example, wrapping each test in a transaction that is rolled back).

9 Useful Built-in Fixtures

pytest ships with several fixtures you can use immediately, with no setup of your own.

9.1 tmp_path: A Temporary Directory

Tests that read or write files should never touch real project files or a shared location. `tmp_path` provides a fresh, unique temporary directory (as a `pathlib.Path`) for each test, automatically cleaned up afterward.

```
def save_report(directory, filename, content):
    path = directory / filename
    path.write_text(content)
    return path

def test_save_report(tmp_path):
    result_path = save_report(tmp_path, "report.txt", "Sales: $1,200")
    assert result_path.exists()
    assert result_path.read_text() == "Sales: $1,200"
```

9.2 capsys: Capturing Printed Output

When code under test writes to standard output (via `print()`), `capsys` lets a test capture and assert on that output.

```
def print_receipt(total):
    print(f"Total due: ${total:.2f}")

def test_print_receipt(capsys):
    print_receipt(19.5)
    captured = capsys.readouterr()
    assert captured.out == "Total due: $19.50\n"
```

9.3 monkeypatch: Safely Patching Things

`monkeypatch` is pytest's built-in tool for temporarily changing attributes, dictionary entries, environment variables, or `sys.path` for the duration of a single test — and it automatically undoes every change when the test ends, even if the test fails.

```
import os

def get_api_key():
    return os.environ["PAYMENT_API_KEY"]

def test_get_api_key(monkeypatch):
    monkeypatch.setenv("PAYMENT_API_KEY", "test-key-123")
    assert get_api_key() == "test-key-123"
```

Key Idea

monkeypatch and unittest.mock.patch() (from the MagicMock reading) solve overlapping problems — both temporarily replace something during a test. As a rule of thumb: reach for monkeypatch for simple attribute, environment-variable, or dictionary substitutions.

10 Parametrized Tests

Recall calculate_late_fee() from the *Testing Foundations* reading, and the negative-input fault discussed there:

```
def calculate_late_fee(days_late, daily_rate=1.50, max_fee=25.00):
    fee = days_late * daily_rate
    if fee > max_fee:
        return max_fee
    return fee
```

Testing several inputs one at a time means writing (and maintaining) a separate test function for each case:

```
def test_typical_fee():
    assert calculate_late_fee(3) == 4.5

def test_fee_is_capped():
    assert calculate_late_fee(30) == 25.00
```

@pytest.mark.parametrize lets you express the same set of cases as data, running the test body once per case while reporting each case as its own pass/fail result:

```
import pytest

@pytest.mark.parametrize(
    "days_late, expected_fee",
    [
        (0, 0.0),          # no late fee
        (3, 4.5),          # typical case
        (30, 25.00),       # fee should be capped
        (100, 25.00),      # far past the cap, still capped
    ],
)
def test_calculate_late_fee(days_late, expected_fee):
    assert calculate_late_fee(days_late) == expected_fee
```

Running this produces four independent test results, one per row of the table, each reported separately in the output:

```
test_calculate_late_fee[0-0.0] PASSED
test_calculate_late_fee[3-4.5] PASSED
test_calculate_late_fee[30-25.0] PASSED
test_calculate_late_fee[100-25.0] PASSED
```

so a single bad case fails on its own, without hiding the fact that the other three still pass.

Tip

Parametrization is an excellent way to encode boundary-value and equivalence-partition test cases (a technique you will study more formally in a later reading) directly into a single, readable test, instead of scattering nearly-identical test functions throughout a file.

The negative-day fault from the *Testing Foundations* reading fits naturally into the same table, documenting the requirement even while it currently fails:

```
@pytest.mark.parametrize(
    "days_late, expected_fee",
    [
        (0, 0.0),
        (3, 4.5),
        (30, 25.00),
        pytest.param(-4, 0.0, marks=pytest.mark.xfail(reason="known bug: see fault in Sec
. 3")),
    ],
)
def test_calculate_late_fee(days_late, expected_fee):
    assert calculate_late_fee(days_late) == expected_fee
```

`pytest.mark.xfail` marks a test case as “expected to fail.” It still runs, but a failure is reported as **XFAIL** rather than **FAIL** — a useful way to document a known defect in the test suite itself without breaking the build, until the underlying fault is fixed.

11 Sharing Fixtures Across Files: `conftest.py`

Copy-pasting the same fixture into every test file quickly becomes unmaintainable. `pytest` automatically discovers fixtures defined in a file named `conftest.py`, without any import statement, and makes them available to every test file in the same directory *and its subdirectories*.

```
tests/
  conftest.py          # fixtures shared by everything under tests/
  test_cart.py
  test_pricing.py
  checkout/
    conftest.py        # fixtures shared only within tests/checkout/
    test_checkout.py
```

```
# tests/conftest.py
import pytest

@pytest.fixture
def sample_cart():
    return {
```

```

    "items": [
        {"name": "widget", "price": 9.99, "quantity": 3},
    ]
}

```

```

# tests/test_pricing.py
# No import of sample_cart needed -- pytest finds it via conftest.py
def test_cart_total(sample_cart):
    assert cart_total(sample_cart) == pytest.approx(29.97)

```

Key Idea

`conftest.py` files form a hierarchy: fixtures defined closer to a test file (in the same directory, or a parent directory) are visible to it; fixtures in a sibling directory are not. This lets you scope shared setup exactly as broadly (or narrowly) as it should be — project-wide fixtures at the top of `tests/`, feature-specific fixtures in a subdirectory.

12 Organizing a Growing Test Suite

The conventions below are not enforced by `pytest`, but they are widely followed in real projects, and following them makes a suite easier for a team to navigate.

- **Mirror your source layout.** If your application code lives in `src/billing/invoices.py`, put its tests in `tests/billing/test_invoices.py`. Anyone can find the tests for a module by following the same path.
- **One test file per source module**, generally. This keeps individual files a manageable size and makes it obvious where a new test for a given piece of functionality belongs.
- **Name tests for behavior, not implementation.** A name like `test_fee_is_capped()` tells you what broke when it fails; a name like `test_function_2()` does not.
- **Keep unit, integration, and slow/system tests distinguishable**, whether by directory (`tests/unit/`, `tests/integration/`) or by marker (below), so a developer can run the fast unit suite constantly and reserve slower tests for CI or pre-merge checks.

12.1 Markers and Selective Test Runs

`pytest` lets you tag tests with custom **markers** and then run only a subset:[8]

```

import pytest

@pytest.mark.slow
def test_full_data_import():
    ... # takes 45 seconds; hits a real (test) database

@pytest.mark.unit
def test_calculate_late_fee():
    assert calculate_late_fee(3) == 4.5

```

Custom markers should be registered (usually in `pyproject.toml` or `pytest.ini`) so `pytest` does not warn that they are unknown:

```
# pyproject.toml
[tool.pytest.ini_options]
markers = [
    "slow: marks tests that are slow to run",
    "unit: marks fast, isolated unit tests",
]
```

Then, from the command line:

```
pytest -m "unit"          # run only tests marked @pytest.mark.unit
pytest -m "not slow"      # run everything except tests marked slow
```

pytest also has two built-in markers worth knowing: `@pytest.mark.skip(reason="...")` unconditionally skips a test, and `@pytest.mark.skipif(condition, reason="...")` skips it only when a condition holds (for example, skipping a test that requires a GPU when none is available).

13 Common Pitfalls

Common Pitfall

Fixtures that are too broad in scope. A session-scoped fixture shared by 200 tests, where a handful of tests mutate it, creates order-dependent failures that are painful to debug. Default to function scope; widen it only when you have measured a real performance problem, and make sure the fixture's state cannot leak between tests when you do.

Common Pitfall

Assertions with no diagnostic value. `assert result` tells you a test failed, but not why. Prefer `assert result == expected_value` so pytest's assertion rewriting can show you both sides of the comparison.

Common Pitfall

Tests that depend on execution order. A test suite should give the same results no matter what order the tests run in (pytest does not guarantee to run them in file order, especially with plugins like `pytest-xdist` for parallel execution). If test B only passes because test A happened to run first and left behind some state, that is a hidden dependency waiting to break.

14 A Practical Mental Model

When you see:

```
def test_something(some_name):
```

think:

“pytest is about to look for a fixture named `some_name` and inject whatever it returns (or yields) into this test.”

When you see:

`yield` inside a fixture

think:

“Everything before this line is setup; everything after it is teardown, and the teardown will run even if the test fails.”

When you see:

```
@pytest.mark.parametrize(...)
```

think:

“This one test function is actually several independent test cases, each reported separately.”

When you see:

a fixture in `conftest.py`

think:

“This is shared setup, visible to every test file in this directory and its subdirectories — no import required.”

15 Summary

The most important `pytest` concepts are summarized below.

Concept	Purpose
Plain <code>assert</code>	Write ordinary assertions; <code>pytest</code> auto-generates detailed failure reports
<code>pytest.approx()</code>	Safely compare floating-point numbers
<code>pytest.raises()</code>	Assert that a specific exception is raised
<code>@pytest.fixture</code>	Define reusable setup, injected into tests by parameter name
<code>yield</code> in a fixture	Split a fixture into a setup phase and a teardown phase
Fixture scope	Control how often a fixture is recreated (<code>function/class/module/session</code>)
<code>tmp_path</code>	A fresh, auto-cleaned temporary directory per test
<code>capsys</code>	Capture and assert on printed output
<code>monkeypatch</code>	Temporarily patch attributes, env vars, or dict entries, auto-reverted
<code>@pytest.mark.parametrize</code>	Run one test body against many input/expected-output cases
<code>conftest.py</code>	Share fixtures across test files without importing them
Markers (<code>@pytest.mark.x</code>)	Tag tests for selective execution (<code>-m</code>), skipping, or <code>xfail</code>

The central idea is simple:

Key Idea

Write tests as plain, readable Python; let pytest handle the machinery. Fixtures replace rigid setup/teardown boilerplate with small, composable, dependency-injected functions; assertion rewriting replaces a zoo of assert-methods with ordinary `assert` statements; markers and `conftest.py` let a suite scale from a handful of tests to thousands without becoming unmanageable.

16 Practice Exercises

Exercise 1 — Assertions

Write a `pytest` test for a function `is_valid_username(username)` that should return `True` for usernames between 3 and 20 characters (inclusive) and `False` otherwise. Include at least one boundary case (exactly 3 characters, exactly 20 characters).

Exercise 2 — Fixtures and Teardown

Write a `yield`-based fixture called `temp_log_file` that creates a file named `app.log` inside `tmp_path`, writes one line to it, yields its path to the test, and then deletes the file during teardown. Write a test that uses this fixture to check the file's contents, and confirm (by reasoning, not just running it) that the file is gone after the test completes.

Exercise 3 — Fixture Scope

A team writes a `session`-scoped fixture that connects to a shared in-memory cache and is used by 50 tests, several of which write new keys into the cache. Explain, in your own words, what could go wrong, and propose two different fixes.

Exercise 4 — Parametrize

Given a function `classify_bmi(bmi)` that returns `"underweight"`, `"normal"`, `"overweight"`, or `"obese"` based on standard BMI ranges, write a single parametrized test covering at least one value from each category and at least one boundary value between two categories.

Exercise 5 — Design Question

A teammate proposes making every fixture in the project `session`-scoped “for speed.” Using at least one concept from this reading and one from the *Testing Foundations* reading (for example, the pesticide paradox, or context-dependent testing), write a short argument for or against this proposal.

17 Discussion Questions

1. What concrete advantage does `pytest`'s assertion rewriting give you over the `assertEqual()` family in `unittest`?
2. Why does a `yield` fixture guarantee its teardown code runs even when a test fails, and why does that matter?
3. When would you choose `class` or `module` scope for a fixture instead of the default `function` scope? What risk are you accepting in exchange?
4. Why might a large project keep `conftest.py` files at multiple levels of its `tests/` directory instead of one file at the top?
5. What is the difference between `@pytest.mark.skip` and `@pytest.mark.xfail`? When would each be the right choice?

6. What makes a test “flaky,” and how can fixture design contribute to or prevent flakiness?
7. Why might a team separate “unit” tests from “slow” or “integration” tests using markers or directory structure?
8. How does the Arrange-Act-Assert pattern make a test easier to read and to debug when it fails?

18 Key Takeaways

For software engineering practice, remember the following principles:

1. **Prefer plain assertions.** Let `pytest`’s assertion rewriting do the work of producing a useful failure message.
2. **Fixtures are dependency injection for tests.** Declare what a test needs as a parameter; let `pytest` supply it.
3. **Default to narrow fixture scope.** Widen scope deliberately, and only when the fixture’s state cannot leak between tests.
4. **Clean up reliably with `yield`.** Anything that must be closed, deleted, or disconnected belongs after a fixture’s `yield`.
5. **Parametrize instead of duplicating.** A table of inputs and expected outputs is easier to read, extend, and review than a wall of near-identical test functions.
6. **Organize for your teammates, not just yourself.** Mirror your source layout, name tests by behavior, and use `conftest.py` and markers so a suite of thousands of tests is still navigable.
7. **A test suite is a codebase.** Apply the same care to its structure, naming, and readability that you apply to production code.

19 References

- Definitions and properties of unit testing.[2, 3, 4]
- `pytest` documentation: Assertions, fixtures, test discovery, and markers.[5, 6, 7, 8]
- Python documentation: `unittest` testing framework.[9]

References

- [1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>
- [2] Fowler, M. (2014, revised 2017). *UnitTest*. <https://martinfowler.com/bliki/UnitTest.html>
- [3] International Software Testing Qualifications Board (ISTQB). (2023). *Certified Tester Foundation Level (CTFL) Syllabus, v4.0*. <https://istqb.org/certifications/certified-tester-foundation-level-ctfl-v4-0/>
- [4] Ottinger, T., & Langr, J. (2009). *F.I.R.S.T.* Agile in a Flash. <http://agileinaflash.blogspot.com/2009/02/first.html>
- [5] `pytest` Development Team. (2026). *pytest Documentation*. <https://docs.pytest.org/en/stable/>
- [6] `pytest` Development Team. (2026). *pytest Documentation: Assertions*. <https://docs.pytest.org/en/stable/how-to/assert.html>
- [7] `pytest` Development Team. (2026). *pytest Documentation: Fixtures*. <https://docs.pytest.org/en/stable/how-to/fixtures.html>

- [8] pytest Development Team. (2026). *pytest Documentation: Markers*. <https://docs.pytest.org/en/stable/how-to/mark.html>
- [9] Python Software Foundation. (2026). *unittest — Unit Testing Framework*. <https://docs.python.org/3/library/unittest.html>